Institut National Universitaire Champollion

Réseaux 3

L3 INFO S5

Algorithme AES :

description détaillée et fondements mathématiques

Institut National Universitaire Champollion

Table des matières

1 Introduction . . . 4
2 Représentation du bloc AES . . . 4
3 Fondements mathématiques : les corps finis . . . 4
3.1 Le corps $GF(2)$ . . . 5
3.2 Représentation d'un octet comme polynôme . . . 5
3.3 Construction de $GF(2^8)$ . . . 5
4 Structure générale d'AES . . . 7
5 Pourquoi $N_r + 1$ clés de tour ? . . . 8
6 Taille des clés de tour . . . 8
7 Paramètres N_b, N_k et N_r . . . 9
7.1 N_b . . . 9
7.2 N_k . . . 9
7.3 N_r . . . 9
8 Nombre total de mots générés . . . 9
8.1 AES-128 . . . 10
8.2 AES-192 . . . 10
8.3 AES-256 . . . 10
9 Les quatre transformations élémentaires d'AES . . . 10
10 SubBytes . . . 10
11 ShiftRows . . . 12
12 MixColumns . . . 12
13 AddRoundKey . . . 14
14 Confusion et diffusion . . . 14
14.1 Confusion . . . 14
14.2 Diffusion . . . 14
15 Déroulement complet d'AES-128 . . . 14
15.1 Initialisation . . . 15
15.2 Tours 1 à 9 . . . 15
15.3 Tour 10 . . . 15
16 Key Expansion (création des clés) . . . 15
17 RotWord . . . 16
18 SubWord . . . 16
19 Rcon . . . 16
20 AES-128, AES-192 et AES-256 . . . 17
21 AES-192 . . . 18
22 AES-256 . . . 18
23 Réversibilité d'AES . . . 19
23.1 SubBytes . . . 19
23.2 ShiftRows . . . 19
23.3 MixColumns . . . 19

23.4 AddRoundKey 19
24 AES comme composition de fonctions 19
25 Vue globale 20

1. Introduction

AES (*Advanced Encryption Standard*) est un algorithme de chiffrement symétrique par blocs.
AES possède :

- une taille de bloc fixe de 128 bits ;
- trois tailles de clé possibles :
  - AES-128 : 128 bits ;
  - AES-192 : 192 bits ;
  - AES-256 : 256 bits ;
- un nombre de tours dépendant de la taille de la clé.

Algorithme	Taille de la clé maître $\mathbf{K}$	Taille du bloc	Nombre de tours N_r
AES-128	128 bits	128 bits	10
AES-192	192 bits	128 bits	12
AES-256	256 bits	128 bits	14

Un point fondamental est que la taille de la clé et la taille du bloc sont deux paramètres indépendants.
La taille du bloc est toujours :

$$128 \text{ bits} = 16 \text{ octets}.$$

En revanche, la clé peut contenir 16, 24 ou 32 octets.

2. Représentation du bloc AES

AES travaille principalement avec des octets.
Un bloc de 128 bits contient :

$$\frac{128}{8} = 16 \; octets.$$

Ces 16 octets sont représentés sous la forme d'une matrice 4×4 appelée *State*.
Si le bloc contient les octets

$$s_0, s_1, \ldots, s_{15},$$

alors AES les place dans la matrice S :

$$\boxed{\begin{pmatrix} s_0 & s_4 & s_8 & s_{12} \\ s_1 & s_5 & s_9 & s_{13} \\ s_2 & s_6 & s_{10} & s_{14} \\ s_3 & s_7 & s_{11} & s_{15} \end{pmatrix}}$$

AES utilise donc un remplissage par colonnes. On appelle cette matrice l'état interne :

$$S = \boxed{State}$$

3. Fondements mathématiques : les corps finis

Les fondements mathématiques d'AES reposent principalement sur l'utilisation de corps finis, en particulier $GF(2)$ et $GF(2^8)$.

3.1. Le corps $GF(2)$

Le corps $GF(2)$ contient seulement deux éléments :

$$GF(2) = \{0, 1\}.$$

Les opérations sont effectuées modulo 2.
Ainsi :

$$0 + 0 = 0,$$

$$0 + 1 = 1,$$

$$1 + 0 = 1,$$

et surtout :

$$1 + 1 = 0.$$

Cette dernière propriété explique pourquoi l'addition binaire dans AES correspond au XOR.

3.2. Représentation d'un octet comme polynôme

Un octet contient 8 bits :

$$b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0.$$

On peut interpréter cet octet comme le polynôme :

$$b_7 x^7 + b_6 x^6 + b_5 x^5 + b_4 x^4 + b_3 x^3 + b_2 x^2 + b_1 x + b_0.$$

Par exemple :

$$\texttt{0x57} = 01010111_2.$$

On obtient donc :

$$\texttt{0x57} \longleftrightarrow x^6 + x^4 + x^2 + x + 1.$$

Les coefficients appartiennent à $GF(2)$.

3.3. Construction de $GF(2^8)$

AES travaille dans le corps fini :

$$GF(2^8).$$

Ce corps contient :

$$2^8 = 256$$

éléments.
AES construit ce corps à partir du polynôme irréductible :

$$m(x) = x^8 + x^4 + x^3 + x + 1.$$

En représentation hexadécimale, ce polynôme correspond à :

$$\boxed{\texttt{0x11B}}.$$

On peut donc écrire :

$$\boxed{GF(2^8) = GF(2)[x]/(x^8 + x^4 + x^3 + x + 1)}$$

Chaque élément de $GF(2^8)$ correspond ainsi à un octet.
.

Addition dans $GF(2^8)$

L'addition dans $GF(2^8)$ correspond au XOR.
Ainsi :

$$a + b = a \oplus b.$$

Par exemple :

$$57 \oplus 83 = \texttt{D4}.$$

Cette propriété provient directement de l'arithmétique dans $GF(2)$, où :

$$1 + 1 = 0.$$

L'addition s'effectue donc bit par bit sans retenue.

Multiplication dans $GF(2^8)$

La multiplication dans $GF(2^8)$ s'effectue en deux étapes :

1. multiplication des polynômes ;
2. réduction modulo
$$m(x) = x^8 + x^4 + x^3 + x + 1.$$

Par exemple :

$$57 \times 83 = \texttt{C1}$$

dans $GF(2^8)$.
Cette multiplication est notamment utilisée dans la transformation `MixColumns`.

Multiplication par x (ou par deux...)

La multiplication par x correspond à un décalage vers la gauche.
Par exemple :

$$57 = 01010111_2$$

donne :

$$01010111 \longrightarrow 10101110$$

soit :

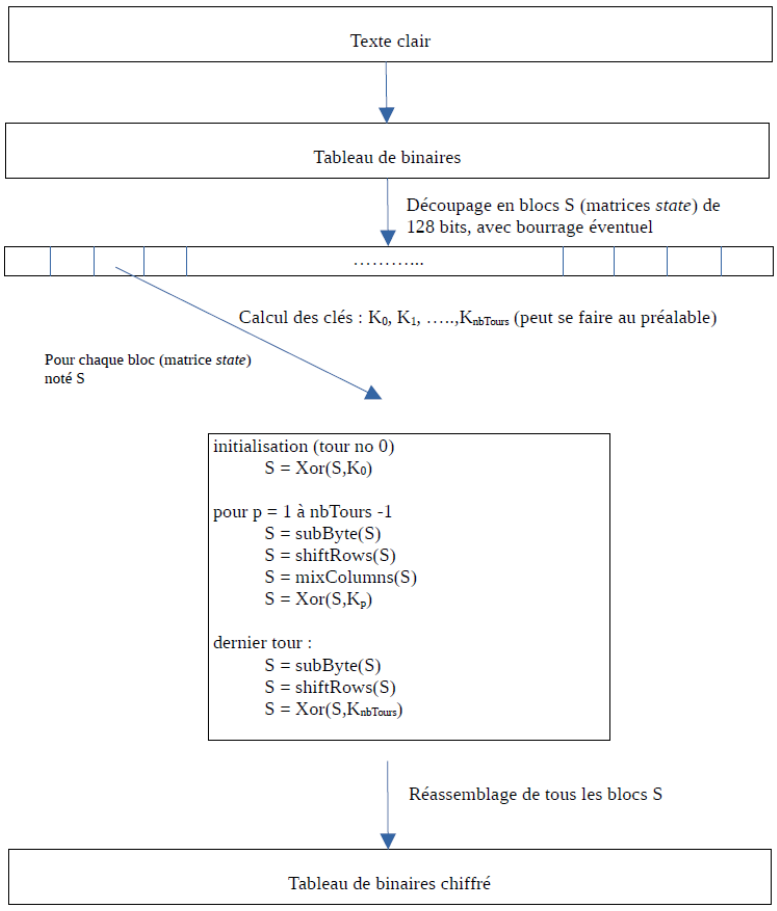
$$\texttt{AE}.$$

Cependant, si le bit de poids fort est égal à 1, une réduction modulo le polynôme irréductible est nécessaire.
Cette opération est souvent appelée :

$$\boxed{xtime()}$$

elle se ramène à faire un Xor avec le binaire b = 00011011 après avoir fait le décalage à gauche.

4. Structure générale d'AES



AES commence par une opération particulière :

$$\boxed{S = S \oplus K_0}$$

appelée aussi AddRoundKey.

Puis plusieurs tours sont exécutés.
Pour AES-128 :

$$N_r = 10.$$

On effectue :

$$\boxed{\text{Initialisation} \rightarrow 9 \text{ tours} \rightarrow \text{tour final}}$$

Les tours classiques contiennent :

$$SubBytes \rightarrow ShiftRows \rightarrow MixColumns \rightarrow AddRoundKey(Xor).$$

Le dernier tour est particulier :

$$SubBytes \rightarrow ShiftRows \rightarrow AddRoundKey(Xor).$$

Il ne contient donc pas de transformation $MixColumns$.

Le calcul des clés K_i se fait à partir de la clé maître K.

5. Pourquoi $N_r + 1$ clés de tour ?

Pour AES-128 :

$$N_r = 10.$$

Il faut donc :

$$N_r + 1 = 11$$

clés de tour.
La première clé est utilisée avant le premier tour (initialisation) :

$$State = S = S \oplus K_0.$$

Puis chaque tour utilise une clé supplémentaire :

$$K_1, K_2, \ldots, K_{10}.$$

On obtient donc :

$$\boxed{K_0, K_1, \ldots, K_{10}}$$

soit 11 clés de tour.
De même :

$$AES\text{-}192 : \quad N_r = 12 \quad \Rightarrow \quad 13 \text{ clés de tour,}$$

et :

$$AES\text{-}256 : \quad N_r = 14 \quad \Rightarrow \quad 15 \text{ clés de tour.}$$

6. Taille des clés de tour

Un point important est que chaque clé de tour K_i fait toujours 128 bits.
En effet, l'état AES contient toujours :

$$4 \times 4 = 16$$

octets, soit :

$$16 \times 8 = 128 \text{ bits.}$$

L'opération $AddRoundKey$ (Xor) effectue :

$$S' = S \oplus K_i.$$

La clé de tour doit donc avoir la même taille que l'état :

$$\boxed{|K_i| = 128 \text{ bits}, \ i = 0, \ldots, N_r.}$$

Cela reste vrai pour AES-128, AES-192 et AES-256.

7. Paramètres N_b, N_k et N_r

AES utilise trois paramètres importants.

7.1. N_b

N_b représente le nombre de mots de 32 bits constituant un bloc.
Comme le bloc fait 128 bits :

$$N_b = \frac{128}{32} = 4.$$

Cette valeur est toujours :

$$\boxed{N_b = 4}.$$

7.2. N_k

N_k représente le nombre de mots de 32 bits constituant la clé.
Pour AES-128 :

$$N_k = \frac{128}{32} = 4.$$

Pour AES-192 :

$$N_k = \frac{192}{32} = 6.$$

Pour AES-256 :

$$N_k = \frac{256}{32} = 8.$$

Ainsi :

$$\boxed{N_k = \begin{cases} 4 & \text{AES-128} \\ 6 & \text{AES-192} \\ 8 & \text{AES-256} \end{cases}}$$

7.3. N_r

Le nombre de tours vaut :

$$\boxed{N_r = \begin{cases} 10 & \text{AES-128} \\ 12 & \text{AES-192} \\ 14 & \text{AES-256.} \end{cases}}$$

8. Nombre total de mots générés

La Key Expansion (génération des clés de tours K_i i = 0,...,N_r) génère :

$$N_b(N_r + 1)$$

mots de 32 bits :

8.1. AES-128

$$4(10+1) = 44.$$

Il y a donc :

$$44$$

mots de 32 bits.
Ces mots sont regroupés quatre par quatre :

$$\frac{44}{4} = 11$$

clés de tour.

8.2. AES-192

$$4(12+1) = 52.$$

Donc :

$$\frac{52}{4} = 13$$

clés de tour.

8.3. AES-256

$$4(14+1) = 60.$$

Donc :

$$\frac{60}{4} = 15$$

clés de tour.

9. Les quatre transformations élémentaires d'AES

AES repose principalement sur quatre transformations :

1. $SubBytes$
2. $ShiftRows$
3. $MixColumns$
4. $AddRoundKey$

10. SubBytes

La transformation $SubBytes$ remplace indépendamment chaque octet de l'état par un autre octet grâce à la S-box.
La S-box AES n'est pas une simple table choisie arbitrairement. Elle est construite mathématiquement à partir de l'inverse multiplicatif dans $GF(2^8)$ suivi d'une transformation affine.
Pour un élément a de $GF(2^8)$:

$$a \longrightarrow a^{-1}.$$

Par convention :

$$0^{-1} = 0.$$

On applique ensuite une transformation affine :

$$b = Aa + c.$$

Ainsi :

$$\boxed{S(a) = A(a^{-1}) + c}$$

où A est une matrice 8×8 sur $GF(2)$ et c est un vecteur constant.
Cette transformation introduit la non-linéarité fondamentale d'AES.
De façon pratique, chaque octet de S est remplacé à l'aide d'une table SBox (donnée ici en base 10) :

S-Box pour AES

S(95)=42=0x2a

	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
00	99	124	119	123	242	107	111	197	48	1	103	43	254	215	171	118
01	202	130	201	125	250	89	71	240	173	212	162	175	156	164	114	192
02	183	253	147	38	54	63	247	204	52	165	229	241	113	216	49	21
03	4	199	35	195	24	150	5	154	7	18	128	226	235	39	178	117
04	9	131	44	26	27	110	90	160	82	59	214	179	41	227	47	132
05	83	209	0	237	32	252	177	91	106	203	190	57	74	76	88	207
06	208	239	170	251	67	77	51	133	69	249	2	127	80	60	159	168
07	81	163	64	143	146	157	56	245	188	182	218	33	16	255	243	210
08	205	12	19	236	95	151	68	23	196	167	126	61	100	93	25	115
09	96	129	79	220	34	42	144	136	70	238	184	20	222	94	11	219
10	224	50	58	10	73	6	36	92	194	211	172	98	145	149	228	121
11	231	200	55	109	141	213	78	169	108	86	244	234	101	122	174	8
12	186	120	37	46	28	166	180	198	232	221	116	31	75	189	139	138
13	112	62	181	102	72	3	246	14	97	53	87	185	134	193	29	158
14	225	248	152	17	105	217	142	148	155	30	135	233	206	85	40	223
15	140	161	137	13	191	230	66	104	65	153	45	15	176	84	187	22

Pour le décodage on utilise la transformation inverse :

$$sBoxInv = \begin{pmatrix} 82 & 9 & 106 & 213 & 48 & 54 & 165 & 56 & 191 & 64 & 163 & 158 & 129 & 243 & 215 & 251 \\ 124 & 227 & 57 & 130 & 155 & 47 & 255 & 135 & 52 & 142 & 67 & 68 & 196 & 222 & 233 & 203 \\ 84 & 123 & 148 & 50 & 166 & 194 & 35 & 61 & 238 & 76 & 149 & 11 & 66 & 250 & 195 & 78 \\ 8 & 46 & 161 & 102 & 40 & 217 & 36 & 178 & 118 & 91 & 162 & 73 & 109 & 139 & 209 & 37 \\ 114 & 248 & 246 & 100 & 134 & 104 & 152 & 22 & 212 & 164 & 92 & 204 & 93 & 101 & 182 & 146 \\ 108 & 112 & 72 & 80 & 253 & 237 & 185 & 218 & 94 & 21 & 70 & 87 & 167 & 141 & 157 & 132 \\ 144 & 216 & 171 & 0 & 140 & 188 & 211 & 10 & 247 & 228 & 88 & 5 & 184 & 179 & 69 & 6 \\ 208 & 44 & 30 & 143 & 202 & 63 & 15 & 2 & 193 & 175 & 189 & 3 & 1 & 19 & 138 & 107 \\ 58 & 145 & 17 & 65 & 79 & 103 & 220 & 234 & 151 & 242 & 207 & 206 & 240 & 180 & 230 & 115 \\ 150 & 172 & 116 & 34 & 231 & 173 & 53 & 133 & 226 & 249 & 55 & 232 & 28 & 117 & 223 & 110 \\ 71 & 241 & 26 & 113 & 29 & 41 & 197 & 137 & 111 & 183 & 98 & 14 & 170 & 24 & 190 & 27 \\ 252 & 86 & 62 & 75 & 198 & 210 & 121 & 32 & 154 & 219 & 192 & 254 & 120 & 205 & 90 & 244 \\ 31 & 221 & 168 & 51 & 136 & 7 & 199 & 49 & 177 & 18 & 16 & 89 & 39 & 128 & 236 & 95 \\ 96 & 81 & 127 & 169 & 25 & 181 & 74 & 13 & 45 & 229 & 122 & 159 & 147 & 201 & 156 & 239 \\ 160 & 224 & 59 & 77 & 174 & 42 & 245 & 176 & 200 & 235 & 187 & 60 & 131 & 83 & 153 & 97 \\ 23 & 43 & 4 & 126 & 186 & 119 & 214 & 38 & 225 & 105 & 20 & 99 & 85 & 33 & 12 & 125 \end{pmatrix}$$

11. ShiftRows

$ShiftRows$ est une permutation des octets de l'état.
Si l'état initial est :

$$State = \begin{pmatrix} a_0 & a_4 & a_8 & a_{12} \\ a_1 & a_5 & a_9 & a_{13} \\ a_2 & a_6 & a_{10} & a_{14} \\ a_3 & a_7 & a_{11} & a_{15} \end{pmatrix},$$

alors les lignes sont décalées de :

- 0 position pour la ligne 0 ;
- 1 position pour la ligne 1 ;
- 2 positions pour la ligne 2 ;
- 3 positions pour la ligne 3.

On obtient :

$$\boxed{\begin{pmatrix} a_0 & a_4 & a_8 & a_{12} \\ a_5 & a_9 & a_{13} & a_1 \\ a_{10} & a_{14} & a_2 & a_6 \\ a_{15} & a_3 & a_7 & a_{11} \end{pmatrix}}$$

Cette opération ne modifie pas la valeur des octets : elle modifie uniquement leur position.

12. MixColumns

$MixColumns$ transforme chaque colonne de l'état indépendamment.
Une colonne est représentée par :

$$\begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}.$$

Elle est multipliée par la matrice :

$$\boxed{M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}}$$

dans $GF(2^8)$.
Ainsi :

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} = M \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}.$$

Toutes les additions sont des XOR et toutes les multiplications sont effectuées dans $GF(2^8)$.
Par exemple :

$$b_0 = 02a_0 \oplus 03a_1 \oplus 01a_2 \oplus 01a_3.$$

Comme :

$$01a = a,$$

on peut écrire :

$$b_0 = 02a_0 \oplus 03a_1 \oplus a_2 \oplus a_3.$$

De même :

$$b_1 = a_0 \oplus 02a_1 \oplus 03a_2 \oplus a_3,$$

$$b_2 = a_0 \oplus a_1 \oplus 02a_2 \oplus 03a_3,$$

$$b_3 = 03a_0 \oplus a_1 \oplus a_2 \oplus 02a_3.$$

Cette transformation assure une forte diffusion de l'information.

Pour le décodage on utilisera l'inverse de M dans $GF(2^8)$:

$$M^{-1} = \begin{pmatrix} 14 & 11 & 13 & 9 \\ 9 & 14 & 11 & 13 \\ 13 & 9 & 14 & 11 \\ 11 & 13 & 9 & 14 \end{pmatrix}$$

(nombres donnés ici en base 10).
Toutes les multiplications se déduisent de la multiplication par deux et de l'addition.
(par exemple 03.a = 02. $\oplus$ a etc...)

13. AddRoundKey

$AddRoundKey$ est l'opération la plus simple d'AES.
On effectue :

$$\boxed{State' = State \oplus RoundKey}$$

octet par octet.
Par exemple :

$$53 \oplus CA = 99.$$

Cette opération correspond à l'addition dans $GF(2^8)$.
Elle est également sa propre inverse :

$$A \oplus K \oplus K = A.$$

Donc :

$$\boxed{AddRoundKey^{-1} = AddRoundKey.}$$

14. Confusion et diffusion

AES repose sur deux concepts fondamentaux de la cryptographie : la confusion et la diffusion.

14.1. Confusion

La confusion est principalement apportée par :

$$\boxed{SubBytes}$$

La relation entre le texte clair, la clé et le texte chiffré devient non linéaire et complexe.

14.2. Diffusion

La diffusion est principalement apportée par :

$$\boxed{ShiftRows + MixColumns}.$$

Une modification locale d'un octet est progressivement propagée à une grande partie de l'état.

15. Déroulement complet d'AES-128

Soit un bloc de texte clair :

$$P \in \{0,1\}^{128}$$

et une clé :

$$K \in \{0,1\}^{128}.$$

Après expansion de la clé, on dispose de :

$$K_0, K_1, \ldots, K_{10}.$$

15.1. Initialisation

On commence par :

$$\boxed{State = P \oplus K_0.}$$

15.2. Tours 1 à 9

Pour :

$$r = 1, \ldots, 9,$$

on effectue :

$$State = AddRoundKey(MixColumns(ShiftRows(SubBytes(State))), K_r).$$

On peut également écrire :

$$State_r = AddRoundKey_{K_r} \circ MixColumns \circ ShiftRows \circ SubBytes(State_{r-1}).$$

15.3. Tour 10

Le dernier tour est :

$$State = AddRoundKey(ShiftRows(SubBytes(State)), K_{10}).$$

Il n'y a pas de $MixColumns$.
Le résultat final est le ciphertext :

$$\boxed{C = State}.$$

16. Key Expansion (création des clés)

La clé maître est découpée en mots de 32 bits :

$$w_0, w_1, w_2, \ldots$$

On définit :

$$N_k = \frac{\text{taille de la clé}}{32}.$$

Ainsi :

$$N_k = \begin{cases} 4 & \text{AES-128} \\ 6 & \text{AES-192} \\ 8 & \text{AES-256.} \end{cases}$$

La clé initiale donne donc :

$$w_0, \ldots, w_{N_k - 1}.$$

Les mots suivants sont générés par récurrence.
Pour :

$$i \geqslant N_k,$$

on pose :

$$temp = w_{i-1}.$$

Si :

$$i \bmod N_k = 0,$$

alors :

$$temp = SubWord(RotWord(temp)) \oplus Rcon[i/N_k].$$

Finalement :

$$\boxed{w_i = w_{i-N_k} \oplus temp.}$$

Cette construction permet de générer toutes les clés de tour.

17. RotWord

Soit un mot de 32 bits :

$$w = \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}.$$

$RotWord$ effectue une rotation circulaire :

$$\boxed{RotWord(w) = \begin{pmatrix} b \\ c \\ d \\ a \end{pmatrix}}$$

18. SubWord

$SubWord$ applique la S-box aux quatre octets d'un mot.
Ainsi :

$$\boxed{SubWord \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} = \begin{pmatrix} S(a) \\ S(b) \\ S(c) \\ S(d) \end{pmatrix}}$$

19. Rcon

$Rcon$ introduit des constantes différentes dans la Key Expansion.
On définit :

$$Rcon[i] = \begin{pmatrix} RC_i \\ 00 \\ 00 \\ 00 \end{pmatrix}.$$

Les premières constantes sont :

$$RC_1 = 01,$$

$$RC_2 = 02,$$

$$RC_3 = 04,$$

$$RC_4 = 08,$$

$$RC_5 = 10,$$

$$RC_6 = 20,$$

$$RC_7 = 40,$$

$$RC_8 = 80,$$

$$RC_9 = 1B,$$

$$RC_{10} = 36.$$

Ces valeurs sont obtenues à partir de puissances de 2 dans $GF(2^8)$, avec réduction modulo le polynôme irréductible d'AES.

20. AES-128, AES-192 et AES-256

Les trois variantes d'AES utilisent le même bloc de 128 bits, mais des tailles de clés différentes.

	Clé	N_k	Bloc	N_r	Mots générés	Clés de tour
AES-128	128 bits	4	128 bits	10	44	11
AES-192	192 bits	6	128 bits	12	52	13
AES-256	256 bits	8	128 bits	14	60	15

Dans les trois cas :

$$N_b = 4.$$

En revanche :

$$N_k = 4, 6, 8.$$

C'est cette différence qui modifie la Key Expansion.

21. AES-192

AES-192 possède une clé de :

$$192 \text{ bits.}$$

Comme un mot contient 32 bits :

$$192 = 6 \times 32.$$

La clé initiale est donc :

$$\boxed{w_0, w_1, w_2, w_3, w_4, w_5}$$

L'expansion génère ensuite :

$$w_6, w_7, \ldots, w_{51}.$$

Les clés de tour sont regroupées par quatre mots :

$$K_0 = (w_0, w_1, w_2, w_3),$$

$$K_1 = (w_4, w_5, w_6, w_7),$$

$$K_2 = (w_8, w_9, w_{10}, w_{11}),$$

et ainsi de suite.
Chaque clé de tour contient toujours :

$$4 \times 32 = 128 \text{ bits.}$$

Ainsi, une clé AES-192 de 192 bits ne produit pas des clés de tour de 192 bits.
Elle est utilisée comme point de départ de la Key Expansion, qui génère une suite de mots de 32 bits regroupés ensuite par quatre.

22. AES-256

AES-256 possède :

$$256 \text{ bits} = 8 \times 32.$$

La clé initiale contient donc :

$$w_0, w_1, \ldots, w_7.$$

La Key Expansion produit ensuite :

$$w_8, w_9, \ldots, w_{59}.$$

AES-256 possède une règle supplémentaire.
Lorsque :

$$i \bmod N_k = 4,$$

on applique :

$$\boxed{temp = SubWord(temp).}$$

Cette étape supplémentaire distingue notamment l'expansion d'AES-256 de celles d'AES-128 et AES-192.

23. Réversibilité d'AES

Chaque transformation utilisée dans AES doit être inversible.

23.1. SubBytes

La S-box est bijective :

$$\boxed{S^{-1}(S(x)) = x.}$$

23.2. ShiftRows

$ShiftRows$ est une permutation des octets. Toute permutation est inversible.

23.3. MixColumns

La matrice :

$$M = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$$

est inversible dans $GF(2^8)$.
Il existe donc une matrice M^{-1} telle que :

$$M^{-1}M = I.$$

23.4. AddRoundKey

Comme :

$$A \oplus K \oplus K = A,$$

on obtient :

$$\boxed{AddRoundKey^{-1} = AddRoundKey.}$$

24. AES comme composition de fonctions

Un tour classique d'AES peut être vu comme une composition de fonctions.
On définit :

$$T_K = AddRoundKey_K \circ MixColumns \circ ShiftRows \circ SubBytes.$$

Le chiffrement complet peut alors être considéré comme une composition des transformations associées aux différentes clés de tour :

$$AES_K = T_{K_{N_r}} \circ \cdots \circ T_{K_1} \circ T_{K_0},$$

avec la particularité que le premier et le dernier tour ont une structure différente.
De manière conceptuelle :

$$\boxed{\text{Clé} \rightarrow \text{Key Expansion} \rightarrow K_0, K_1, \ldots, K_{N_r}}$$

et :

$$\boxed{\text{Plaintext} \to SubBytes \to ShiftRows \to MixColumns \to AddRoundKey \to \cdots \to \text{Ciphertext.}}$$

25. Vue globale

AES peut finalement être compris comme la combinaison de quatre mécanismes fondamentaux :

$$\boxed{\text{Non-linéarité} + \text{Permutation} + \text{Diffusion} + \text{Dépendance à la clé}}$$

Les transformations jouent les rôles suivants :

Transformation	Fonction principale
$SubBytes$	Non-linéarité
$ShiftRows$	Permutation
$MixColumns$	Diffusion
$AddRoundKey$	Incorporation de la clé